

Case Studies & Build Logs

This chapter records build logs at multiple resolutions, from function-level specifications to gate-level realizations, timing behavior, and device-level constraints. The organizing idea is the error profile $\varepsilon(r)$, viewed as a resolution-dependent discrepancy measure that can be tracked across successive refinement steps. In practice, the same design may be inspected as a Boolean function, a gate network, a timing-constrained netlist, or a physical device artifact; the build log should preserve the correspondence among these views so that regressions, optimizations, and closure failures can be localized precisely.

Multi-resolution dashboards. A useful build dashboard exposes at least four linked layers: function, gate, timing, and device. At the function layer, one records the intended truth table, algebraic normal form, or specification fragment. At the gate layer, one records the chosen decomposition, including shared subexpressions and structural symmetries. At the timing layer, one records critical paths, slack, and any retiming or buffering decisions. At the device layer, one records the implementation substrate and the constraints that arise from it, such as fan-out limits, placement effects, or technology-specific primitives. The purpose of the dashboard is not merely reporting; it is to make refinement steps auditable and to expose where a design ceases to be stable under further composition. In this sense, $\varepsilon(r)$ is not just a score but a diagnostic trace: as the resolution changes, the build log should show which discrepancies disappear, which persist, and which are introduced by the act of refinement itself.

From NAND to a 4-bit ALU to a microcontroller. The chapter uses a progression of increasingly rich artifacts as a recurring case-study pattern. The first stage is a NAND-centered construction, where a small basis is expanded into derived gates and then into larger combinational blocks. The second stage is a 4-bit ALU, where arithmetic and logic operations are assembled from reusable slices and control logic. The third stage is a microcontroller-scale integration, where datapath, control, memory interface, and timing closure interact. Across these stages, the same questions recur: which symmetries survive the refinement, which optimizations preserve semantics, and which implementation choices introduce new constraints that must be tracked in the build log. The point of the progression is not only to scale up complexity, but to show that the same bookkeeping discipline applies from a single gate family to a system-on-chip-style artifact.

Compression by symmetry. A central optimization theme is compression by symmetry, especially through LUT packing and factoring. When a family of subfunctions is related by permutation, complementation, or other structural symmetries, the implementation can often share resources rather than duplicating them. LUT packing exploits the fact that multiple small functions may fit into a single configurable resource when their inputs and outputs are arranged compatibly. Factoring reduces repeated structure by extracting common subexpressions or common control patterns. In both cases, the build log should record the symmetry used, the transformation applied, and the resulting effect on area, delay, and maintainability. The key point is that optimization is not only a local improvement; it is a claim that the compressed artifact remains equivalent to the original specification under the chosen notion of refinement. When the compression is successful, the log should make clear whether the gain came from a true symmetry of the specification or from an incidental regularity of the chosen encoding.

Post-mortems and closure failures. Not every refinement succeeds. This chapter therefore includes post-mortems for closure failures, meaning cases where an intended compositional law or symmetry-preserving transformation does not hold in the implemented artifact. Such failures are valuable because they often produce counterexamples that clarify the boundary of a theorem, a synthesis rule, or a design heuristic. A post-mortem should identify the failed assumption, the smallest counterexample, the resolution at which the failure first appears, and the corrective action, if any. In the context of this book, counterexamples are not merely negative results; they are diagnostic artifacts that refine the closure story by showing exactly where the hierarchy stops being stable. A good post-mortem also distinguishes logical failure from physical failure: some counterexamples arise because the specification was too strong, while others appear only after timing, placement, or device constraints are taken into account.

Artifact index and reproducibility. The chapter also serves as an artifact index. It should point to the repository structure, schema definitions, and toolchain specifications needed to reproduce the builds described in the logs. The repository index identifies where specifications, generated netlists, timing reports, and validation scripts live. The schemas define the expected shape of intermediate artifacts so that logs can be parsed and compared mechanically. The toolchain specification records versions, synthesis settings, and any environment assumptions required for reproducibility. Together, these materials make the chapter a bridge between narrative explanation and executable evidence. A reader should be able to move from the prose description of a build to the exact files and commands that generated it, and then back again through the logged resolutions r and discrepancy values $\varepsilon(r)$.

Open problems. Several open problems remain intentionally visible. How should $\varepsilon(r)$ be normalized across heterogeneous resolutions so that function-level and device-level discrepancies can be compared meaningfully? Which symmetry classes admit the most effective LUT packing without sacrificing timing closure? When does factoring improve area but worsen critical path behavior, and how should such trade-offs be represented in a build log? What is the right abstraction for closure failures that arise only after physical implementation, rather than at the logical or gate level? These questions are left open as prompts for further experimentation and for future artifact-driven refinement. They also mark the boundary between what can be asserted abstractly and what must be established empirically in a concrete toolchain.

Chapter use. Readers may treat this chapter as a logbook template: each case study should state the specification, the refinement path, the optimization or failure observed, and the reproducibility artifacts required to replay the result. In that sense, the chapter is less a single theorem than a disciplined record of how the book’s abstract framework behaves when pushed through concrete builds. The intended workflow is iterative: record the artifact, measure $\varepsilon(r)$, attempt compression or refinement, and then preserve both successes and failures as part of the permanent build history.

Repository and schema placeholders. For reproducibility, the chapter should be paired with a repository index and machine-readable schemas. A minimal index may include entries for specification sources, generated netlists, timing reports, device constraints, and validation scripts. Likewise, the schema layer should describe the expected fields for each artifact class so that build logs can be checked automatically. In the present draft, these references are intentionally left as placeholders to be filled by the project-specific repository layout and toolchain manifest.

Build-log template. A typical entry in the log may record: the source specification, the target resolution r , the transformation applied, the measured discrepancy $\varepsilon(r)$, and any observed trade-off in area, delay, or closure. When a transformation fails, the same template should capture the counterexample, the first failing resolution, and the reason the failure matters for later refinement steps. This template is meant to be reused across the NAND, ALU, and microcontroller case studies so that the chapter reads as a coherent sequence of comparable experiments rather than as isolated anecdotes.